

Radical Denesting

After the First Repair

A review of `StradFixed.wl`

Correctness boundaries, principal branches, resource control,
search completeness, and a certificate-gated replacement

Prepared for Vladimir Reshetnikov
Repository: `VladimirReshetnikov/RadicalDenest`
Audit date: 5 September 2026

Principal conclusion. The repaired implementation has a sound central idea: generate candidate radical expressions and compare them with the *original selected algebraic value*. That safeguard is not applied uniformly at the wrapper boundary. In particular, a user-supplied solver can corrupt a symbolic host. The advertised time and multiplier limits also do not bound all relevant work.

The accompanying `StradImproved.wl` is a conservative standalone proposal, not a claim of a fully tested production replacement. It centralizes candidate acceptance, shares request budgets, streams capped multiplier proposals, and adds elementary fourth-root and cubic-in-quadratic methods.

Validation status. Independent Python/SymPy checks were executed: 33 named checks, including 312 rational parameter cases. A Wolfram regression suite and runner are supplied but were *not executed*: both native-kernel connector attempts failed, and no local Wolfram kernel was present. A lexical delimiter check is not a Wolfram syntax or semantics test.

The source revision was inspected through the public mutable `main` branch. No commit identifier or original-file byte hash could be obtained. Access limitations and their consequences are recorded in Section 1.

Contents

1	Scope, evidence, and overall assessment	1
1.1	What was inspected, and what was not	1
1.2	Evidence terminology	1
1.3	What the first repair gets right	2
2	The mathematical contract comes before the algorithm	2
2.1	Values, presentations, and branches	2
2.2	A representation cost with an explicit domain	3
2.3	The exact-certificate boundary	3
3	The multiplier–polynomial–orbit architecture	4
3.1	A precise justification of the orbit step	4
3.2	A fully explicit test of the mechanism	4
3.3	What multiplier degrees do not prove	5
4	Prioritized findings	5
4.1	F01: a symbolic host escapes the central guarantee	6
4.2	F02: assumptions and expression semantics are underspecified	7
4.3	F03: a deadline check is not a whole-computation limit	7
4.4	F04: the cap is neither a cardinality invariant nor a construction bound	8
4.5	F05: invalid controls can be worse than absent controls	8
4.6	F06: effective options and explicit options are different	9
4.7	F07: visited state loses the history of skipped work	9
4.8	F08: candidate existence is not successful improvement	9
4.9	F09: the final candidate, not its ancestor, needs certification	10
4.10	F10: numerical rejection is not an exact proof of inequality	10
4.11	F11: a local factorization guard does not prove the emitted rewrite	10
4.12	F12–F16: remaining contract and coverage limitations	11
5	Algebraic steps that should not be “fixed” incorrectly	12
5.1	The direction of Order	12
5.2	The nested-power guard has a valid core	12
5.3	The denominator formula has the correct sign	12

5.4	Numerical scheduling is not automatically unsound	13
6	Low-cost methods worth adding first	13
6.1	Direct quadratic denesting	13
6.2	Indirect fourth-root denesting	13
6.3	Complex quadratic constants	14
6.4	A cube root inside a quadratic field	14
6.5	Rational numerators and future field-specific producers	15
7	The proposed replacement: design and compatibility	15
7.1	Structure and trust boundaries	16
7.2	The queue and the units of its limits	16
7.3	Option reference	16
7.4	Commit and rollback policy	17
7.5	Result interpretation and usage	18
8	Verification, regression tests, and reproducibility	18
8.1	What actually ran	19
8.2	Native integration suite: pending, not passed	19
8.3	A benchmark corpus that distinguishes tasks	20
9	A practical improvement roadmap	20
9.1	First: establish a trustworthy acceptance boundary	20
9.2	Second: improve algorithms without widening the trust boundary	20
9.3	Third: make results independently auditable	21
10	Conclusion	21
A	Archive inventory and build instructions	21
B	Targeted literature and source references	22

1 Scope, evidence, and overall assessment

This review examines the corrected implementation, not merely the earlier code that motivated its repair. Its objective is to distinguish four questions: whether a returned expression has the right value; whether it is genuinely a radical expression of lower cost; whether the computation respects its controls; and which denestings the search can miss. These are different obligations. A successful exact equality check cannot establish completeness, and a useful degree heuristic cannot replace a branch certificate.

1.1 What was inspected, and what was not

The complete raw web rendering of `src/corrected/StradFixed.wl` was inspected, including its factorizer, marker wrapper, quadratic fast path, multiplier scheduling, polynomial-GCD trials, and discriminant generator. The linked unified research report was consulted in its current TeX rendering, especially the sections on branch models, direct and indirect quadratic denesting, cubic reconstruction, equality, and implementation contracts. The literature-directory README and the preceding code-review-directory README were also accessible. Their directory listings established the intended supporting corpus.[\[1, 2, 3, 4\]](#)

The body files of the preceding unified code review, its detailed experiment logs, and the literature archive's individual catalogue/manifest entries could not be retrieved reliably. Thus this is *not* a claim to have rechecked every assertion or replayed every experiment in that earlier review. Relevant original literature was instead consulted through accessible public copies: BFHT, Jeffrey–Rich, Gkioulekas, Cavallo, and selected passages in Honsbeek. Some PDF screenshot requests failed; the accessible parsed passages, the successful Gkioulekas theorem-page image, and explicit algebraic derivations were used rather than guessing illegible formulas.[\[5, 6, 7, 8, 9\]](#)

The old Library copy of an earlier unified survey was encountered during retrieval, but was not treated as a byte-identical copy of the current repository report. Likewise, a README claim about a previous Wolfram run is not evidence that *this* corrected source or the proposed replacement ran here.

No repository commit could be pinned. The public code page and raw renderer gave different line-count presentations. Findings below therefore use function names and distinctive control-flow locations instead of pretending to provide stable checkout line numbers. The archive contains hashes of the *newly generated deliverables*, not an invented hash of the original source.

1.2 Evidence terminology

S—static finding means a consequence of the inspected source and the language contract, sometimes with an executable reproduction supplied for later native confirmation. **M—mathematical finding** means an explicit algebraic or finite combinatorial argument. **R—runtime-sensitive risk** means a plausible failure mode that has not been demonstrated in a native Wolfram run. **L—scope limitation** means an absent guarantee or a deliberately incomplete strategy, not an incorrect equality.

The strongest findings do not depend on timing a commercial kernel. For example, the wrapper's absence of a certificate on a particular path and a generator returning nine values under a cap of seven are inspectable properties. Nevertheless, a source-implied reproduction is labeled as such, not reported as a measured kernel output.

1.3 What the first repair gets right

The corrected code moves private markers and polynomial variables out of the user's global namespace, avoids modifying `Print`, and uses an explicit cost order. Most importantly, its numerical core compares reconstructed candidates with the original problem rather than only with the principal root of a power of that problem. It also rejects opaque algebraic objects preferentially in its cost and attempts exact verification after normalization. These choices should be retained.[1]

The recommended response is therefore not to discard the algebraic architecture. It is to make certification an unavoidable interface boundary, make resource limits apply before work is materialized, and stop treating search bookkeeping as a substitute for a specified completeness theorem. The accompanying rewrite takes that approach, with explicit compatibility changes rather than a silent claim of drop-in equivalence.

2 The mathematical contract comes before the algorithm

2.1 Values, presentations, and branches

A finite radical expression denotes both a selected algebraic number and a syntax tree. The same value may have many trees. In a principal-power convention,

$$z^{p/q} = \exp\left(\frac{p}{q} \operatorname{Log} z\right), \quad q > 0,$$

for nonzero z , with the usual principal logarithm. In particular a negative real number raised to a fractional power with odd denominator does *not* generally mean its real odd root. Real-root notation must be introduced separately.

An exact algebraic-number object solves a representation problem, not necessarily a denesting problem. Reducing an expression to a `Root` object can make its printed nesting disappear without supplying any radical formula. A trigonometric expression can hide the same issue. The proposed implementation therefore permits `Root` and `AlgebraicNumber` as inputs for attempted conversion, but not as accepted radical outputs. It likewise rejects transcendental-function syntax in proposed numeric outputs, even when a particular such expression is algebraic.

For example, take $\zeta = \exp(\pi i/8)$. Then

$$\sqrt{2 + \sqrt{2}} = \zeta + \zeta^{-1} = (-1)^{1/8} + (-1)^{-1/8}. \quad (1)$$

The last expression is a depth-one *complex-radical* expression with principal powers. Calling $2 \cos(\pi/8)$ a depth-zero radical answer would instead change the output language. The distinction between real and complex radical models is substantive; it is not a cosmetic formatting choice.[5, 7]

For a direct algebraic certificate of the polynomial part of (1), use $\zeta^8 = -1$ and $\zeta^{-1} = -\zeta^7$. The element $y = \zeta - \zeta^7$ satisfies $y^4 - 4y^2 + 2 = 0$. The choice $\operatorname{Arg} \zeta = \pi/8$ gives $y = 2 \cos(\pi/8) > 0$ and $y^2 > 2$, which selects $y = \sqrt{2 + \sqrt{2}}$. The polynomial alone would not select that root.

2.2 A representation cost with an explicit domain

For rational constants, arithmetic, and noninteger rational powers, use

$$\text{depth}(c) = 0, \quad \text{depth}(E \circ F) = \max(\text{depth}(E), \text{depth}(F)), \quad \text{depth}(E^{p/q}) = 1 + \text{depth}(E) \quad (q > 1).$$

Integer powers add no radical level. This is a syntax measure over a specified base field; it is not the degree of the generated number field, the length of a field tower, or a Galois group's derived length.

The original code's actual cost has five components, although its usage text describes four. The replacement defines six nonnegative integer components:

$$C(E) = (O(E), D(E), R(E), L(E), J(E), B(E)). \quad (2)$$

Here O counts opaque algebraic objects, D is radical depth, R counts noninteger rational-power nodes, L is LeafCount, J is the largest root index, and B counts the binary sizes of integer and rational constants under a documented elementary convention. All accepted numeric outputs are already required to have $O = 0$.

The lexicographic order on these tuples is well founded. Consequently a strictly decreasing sequence of committed costs cannot cycle. This is a termination argument for the improvement relation, not a claim that every computational operation terminates quickly, nor that the final expression minimizes C among all equal radical expressions. Explicit iteration, time, memory, and search limits remain necessary.

Giving opaque-object count first means that conversion of a Root input to any admissible bounded radical representation can outrank its original zero displayed radical depth. Giving depth before size allows a larger expression to win when it has smaller depth. Those are intentional policies and should be exposed to future benchmarking rather than described as uniquely correct measures of simplicity.

2.3 The exact-certificate boundary

The proposed trust boundary is: a candidate producer may fail, return a wrong branch, or return a worse expression; it may not directly replace the target. A single acceptance function checks finite size, admissible radical syntax, strict improvement, exact algebraicity, and

$$\text{RootReduce}(F - E) \equiv 0 \quad (3)$$

as an exact zero. Only then is the candidate retained. Wolfram documents RootReduce as canonical algebraic reduction; the use here is equality verification, not a promise of radical reconstruction.[10]

The certificate must refer to the target *before* the transformation. Rechecking only a candidate against an intermediate candidate can perpetuate an earlier error. After polishing, the polished expression is a new candidate and must pass the same boundary.

For pure arithmetic hosts, replacing a fixed numeric value by an exactly equal numeric value is a congruence: addition, multiplication, and a fixed rational-power operation receive the same argument value. This reasoning does not license traversing arbitrary held syntax, user-defined heads, callbacks with effects, or symbolic simplification under implicit assumptions. The replacement intentionally restricts host traversal.

3 The multiplier–polynomial–orbit architecture

3.1 A precise justification of the orbit step

Let $E \neq 0$ be the selected input value, let $r \geq 2$ be an integer, and set $A = E^r$. For a nonzero algebraic multiplier m , form the minimal polynomial $p_m(X)$ of the principal value $(mA)^{1/r}$. Over an algebraic coefficient field containing mA , calculate

$$g_m(X) = \gcd(p_m(X), X^r - mA).$$

A smaller factor can offer easier roots to express. The algebraic reason for the phase orbit is the following.

Proposition 3.1 (Orbit coverage for a selected root). *If u is any root of g_m , then one of*

$$\frac{u}{m^{1/r}} \zeta_r^k, \quad 0 \leq k < r,$$

equals E , where $m^{1/r}$ denotes the principal value and $\zeta_r = \exp(2\pi i/r)$.

Proof. Since g_m divides $X^r - mA$, we have $u^r = mA$. Both u and $m^{1/r}$ are nonzero. Put $v = u/m^{1/r}$. Then $v^r = A = E^r$, so $(E/v)^r = 1$. The quotient E/v is one of the r th roots of unity. No identity distributing a principal fractional power across a product was used. $\square$

This proof explains why the original-value certification in the repaired numerical core is an important improvement. It also explains why comparing r th powers, or checking membership in the same minimal polynomial, cannot choose a branch. The two positive numbers $\sqrt{2} + \sqrt{3}$ and $\sqrt{3} - \sqrt{2}$ have the same polynomial $X^4 - 10X^2 + 1$ but are not equal.

The condition $0 < \deg g_m < r$ is useful for reconstruction but is not itself a denesting certificate. A formula for a root of g_m may still contain coefficients just as deeply nested as the original expression. Final expression cost must decide whether an algebraic success is a presentational improvement.

3.2 A fully explicit test of the mechanism

Set $t = \sqrt[3]{2} > 0$ and $E = \sqrt[3]{t-1} > 0$. Take $r = 3$ and $m = 9$. The element

$$u = 1 - t + t^2$$

satisfies

$$u^3 = 9(t-1), \quad u > 0,$$

because $u = (t - 1/2)^2 + 3/4$. Its minimal polynomial is

$$p(X) = X^3 - 3X^2 + 9X - 9.$$

In $\mathbb{Q}(t)[X]$ the relevant GCD is

$$\gcd(p(X), X^3 - 9(t-1)) = X - (1 - t + t^2).$$

Hence $E = u/\sqrt[3]{9}$ is a depth-one radical expression. The companion SymPy script actually computes this minimal polynomial and field GCD, as well as the polynomial remainder certificate. This is an executed independent test of the mathematics of the architecture, not a run of Wolfram’s PolynomialGCD implementation.

The revised native suite includes a forced-multiplier-9 capability test. Failure of that test on a real kernel would reveal a code, representation, or budget problem; it would not invalidate the identity or establish non-denestability.

3.3 What multiplier degrees do not prove

A minimal-polynomial degree is a property of the represented value, whereas radical depth is a property of a representation language and expression. Degree reduction can make reconstruction cheaper, but the relation is neither an equivalence nor a monotone rule for arbitrary candidate histories. A discriminant-prime search may be effective on examples without containing all multipliers needed for all denestings.

The general literature uses substantially more structure—field extensions, independence, splitting fields, and carefully stated root-of-unity hypotheses—than this heuristic queue does. This implementation is not an implementation of a complete general minimum-depth algorithm simply because it invokes exact polynomial arithmetic. BFHT’s representation-cost distinctions and Honsbeek’s field-specific criteria are particularly relevant cautions.[\[5, 9\]](#)

4 Prioritized findings

The following table is an index; the detailed arguments follow. Priority describes the consequence under the advertised API, not the frequency observed in a benchmark. No native benchmark frequency was measured.

ID	Priority	Evidence	Finding
F01	Critical	S	Custom-solver replacements in symbolic hosts bypass equality certification.
F02	High	S/L	Ambient assumptions and arbitrary host traversal exceed the unconditional numeric-rewrite contract.
F03	High	S	Time controls do not enclose expensive producers; nested and list calls restart budgets.
F04	High	M/S	The multiplier cap is violated, and truncation follows eager exponential construction.
F05	High	S	Option values are not validated; an infinite cap makes the generator’s counting loop unbounded.
F06	Medium	S	Resolved public defaults are not propagated; SetOptions and factor controls have inconsistent scope.
F07	Medium	M/L	A visited entry does not record whether a history-dependent trial stage was executed.
F08	Medium	S/L	A certified candidate can stop search even when it does not improve the incumbent.

ID	Priority	Evidence	Finding
F09	Medium	S/R	Polishing is not rechecked before selecting the winner; conservative fallback can discard valid alternatives.
F10	Medium	R	Fixed-precision rejection is not a justified exact-identity rejection test.
F11	Medium	M/R	Aggregate factor positivity does not establish every per-base power rewrite emitted by the custom factorizer.
F12	Medium	L	Syntax cost does not enforce an output language; hidden-function and opaque-object issues remain.
F13	Medium	L	Cheap fourth-root and cubic-in-quadratic reconstructions are missing from the specialized path.
F14	Medium	S/L	Mixed inexact hosts and all-or-nothing ToRadicals rollback lose independent exact opportunities.
F15	Medium	L	Search controls do not bound algebraic degree, intermediate size, phase-orbit allocation, or duplicate values uniformly.
F16	Low/Med.	S/L	Result status, documentation, magic constants, and reproducibility need stronger contracts.

4.1 F01: a symbolic host escapes the central guarantee

The wrapper substitutes solver outputs into marked numeric subexpressions. It then verifies candidate equality only when the *entire original expression* passes its exact-algebraic numeric predicate. A symbolic host fails that predicate, so no final equality check is performed on the assembled result. The ordinary core certifies its own output, but a user-supplied "Solver" need not do so.[1]

The following is a source-implied counterexample, supplied as a native reproduction rather than a reported native result:

```
Clear[x];
RadicalDenest`Strad[
  x + Sqrt[3 + 2 Sqrt[2]],
  "Solver" -> (0 &), "Factor" -> False
]
(* Source-implied incorrect result: x *)
```

The custom producer replaces the marked numeric radical by zero. The host is not a numeric algebraic number, so the wrapper does not reject the changed value. Removing the radical also improves every relevant cost coordinate, so the cost filter provides no protection.

This is a correctness defect under the unconditional public usage statement, not merely a missing optimization. It is conditional on use of the custom-solver API; it is *not* evidence that every automatic-core result is wrong. The remedy is local certification before substituting each solver output. Requiring external solvers to be trustworthy by convention is weaker than the advertised guarantee and should not be the default.

The replacement applies the same `acceptCandidate` function to custom and built-in proposals. A callback remains trusted *code* in the security sense—it can perform effects or modify definitions—but

its returned expression is not trusted mathematical evidence.

4.2 F02: assumptions and expression semantics are underspecified

The wrapper also applies ordinary symbolic simplification to its assembled candidate. Without an explicit assumptions policy, the ambient `$Assumptions` can affect a symbolic host, while the whole-expression exact-algebraic check is again skipped. For example, a simplifier acting under $x > 0$ may replace $\sqrt{x^2}$ by x . That is valid under the assumption, but not an unconditional identity over all complex substitutions for x .

This is best described as an API-contract gap: assumption-aware simplification could be a legitimate separate feature, but then assumptions must be input data and part of the returned certificate. The proposed code instead blocks ambient assumptions to `True` and does not simplify symbolic hosts globally.

Moreover, algebraic equality of subexpressions is not permission to rewrite arbitrary held syntax. An expression representing a program, a pattern, or a symbolic data structure may distinguish the original syntax from an equal value. The new walker descends only into `Plus`, `Times`, `List`, and `Power` with rational exponent. It leaves unknown heads and held constructs alone. This deliberately narrows traversal; it is not a universal simplifier for arbitrary Wolfram expressions.

The reasoning presumes ordinary pure arithmetic expressions without adversarial `UpValues` or evaluation side effects. Neither version can turn a live Wolfram expression evaluator into a security sandbox merely by checking algebraic equality.

4.3 F03: a deadline check is not a whole-computation limit

The core checks its deadline between multiplier trials, but a single trial may call `MinimalPolynomial`, `PolynomialGCD`, and `Solve` without a surrounding time constraint. Discriminants, integer factorization, lazy-batch materialization, normalization, and candidate products can likewise run before the next deadline test. Public denominator rationalization has no enclosing resource contract.[1]

The certificate routine itself performs algebraicity predicates and numerical approximation before entering its timed exact-normalization region. Its fallback may then use another full allowance. Consequently the option called `"CertifyTime"` is not a bound on the entire predicate, even before considering repeated candidates.

List mapping starts independent calls. Recursive refinement similarly creates fresh cores and fresh deadlines. A nominal budget of T therefore does not mean that a list or a multi-stage wrapper invocation finishes in T seconds. In the worst structural case the nominal budget is multiplied by the number of subcalls, while unguarded individual operations can exceed it independently.

The replacement encloses the request body in `TimeConstrained` and `MemoryConstrained`, shares an absolute deadline and trial counter across traversal and passes, and gives expensive operations the smaller of their operation allowance and remaining request time. The equality predicate's algebraicity checks and subtraction occur inside its allowance. Before committing a new pass, the absolute deadline is checked again.

These remain cooperative kernel controls, not an operating-system service-level guarantee. Official documentation notes timer granularity, the effect of `AbortProtect`, and limitations concerning other processes/threads. A hostile callback or uninterruptible subsystem can defeat a simplistic wall-clock

promise. Production isolation should use a separate kernel process with an external watchdog where hard deadlines matter.[11, 12]

4.4 F04: the cap is neither a cardinality invariant nor a construction bound

The discriminant generator computes how many small primes could fit a nominal product count, but then unions *all* discriminant primes back into its divisor list. It also forces at least one selected prime. Thus the proposed bound does not apply to the emitted batch.

For a concrete helper input, use root index $r = 2$ and the polynomial $X^2 + 2310$. Its discriminant is -9240 , whose distinct positive prime factors are

$$P = \{2, 3, 5, 7, 11\}.$$

At cap 7, the selected first three primes produce the divisors of 30. Union with P and removal of 1 gives

$$\{2, 3, 5, 6, 7, 10, 11, 15, 30\},$$

which has nine elements. At caps 0 and 1, the forced first prime together with that union still gives five elements. These counts were checked by the independent Python model, and the optional native helper reproductions use this explicit polynomial. No hypothetical full search history is needed for the helper-level violation.

The same union reintroduces large primes removed by the preceding ratio-pruning logic. That makes the pruning policy partly ineffective even apart from the cardinality error.

Elsewhere, the core forms products of subsets of promising initial multipliers using an eager distribution operation, then truncates the result. With k binary choices, 2^k combinations can be constructed *before* a small final cap is applied. At $k = 30$, that is more than a billion combinations. Bounding the length of the retained list does not bound peak memory or generation time.

The new queue enforces `Length[queue] <= cap` at every insertion, including initial and user-supplied multipliers. It never constructs a complete divisor lattice or Cartesian product. Its streamed prime-power heuristic is intentionally different and may explore fewer combinations; the gain is a meaningful allocation invariant, not a completeness improvement.

4.5 F05: invalid controls can be worse than absent controls

The original public interfaces do not validate numeric option domains. An infinite "MultiplierCap", for example, leaves the generator's loop asking for ever larger integers satisfying $r^{t+1} - 1 \leq \infty$. There is no mathematical terminal value of t . Negative, fractional, symbolic, or malformed limits can instead trigger inappropriate comparisons, slicing, or silent fallback.

The rewrite rejects unknown keys and invalid domains with `Failure`. Caps and trial counts are finite nonnegative integers; pass, degree, root-index, leaf, and memory limits are positive integers; operation allowances are finite positive real numeric quantities; the total time budget may be zero. Boolean options must be actual `True` or `False`. An invalid multiplier element is conservatively skipped during bounded validation rather than being passed directly into polynomial arithmetic.

A zero total time budget is an immediate no-op. A zero multiplier cap or zero trial budget disables multiplier search but does not disable the inexpensive specialized methods. These distinctions are tested and documented.

4.6 F06: effective options and explicit options are different

The wrapper forwards core options selected from the explicitly supplied rule sequence. It does not first resolve all public defaults. Further, the `Strad` entry delegates rather than reading its own default values. As a consequence, modifying defaults with `SetOptions[Strad, . . . ]` need not affect the delegated computation; modifying a wrapper default for a core-only control can also be lost when no explicit rule is forwarded.

This is especially confusing for safety controls: a user who sets a default trial budget of zero expects that policy to apply to future calls. Passing an explicit rule often works while changing the documented default does not. The proposed entry points resolve their own full defaults once, merge explicit options with first-rule precedence, validate the resulting association, and pass that association through the entire engine.

The original factor toggle has similarly incomplete scope. Disabling the outer factor pass does not prevent `normalizeRadicand` from invoking the custom factorizer, and recursive re-entry can restore wrapper defaults. The replacement defines "Factor" narrowly and explicitly: it controls the optional `FactorTerms` presentation candidate. It does not claim to disable the polynomial factorization used by cubic reconstruction or the integer factorization used to propose discriminant multipliers.

4.7 F07: visited state loses the history of skipped work

A multiplier's minimal-polynomial degree is cached in the visited association, after which repeated multipliers are skipped. But the work done by `tryMultiplier` depends on the current best degree and another history flag. A cached degree alone does not say whether a GCD was computed, whether reconstruction ran, or whether the trial was timed out or filtered.

The gate contains a condition equivalent to

$$d \leq b \quad \text{or} \quad \gcd(d, b) < b.$$

For $d = 12, b = 6$ it is false; for $d = 12, b = 5$ it is true. Thus a multiplier seen in the former state can be incorrectly regarded as fully explored in the latter state. This is an exact control-flow countermodel, independently checked in Python. It is *not* a claimed concrete radical input that has been observed to realize the complete history.

A sound cache design separates invariant facts (canonical multiplier, polynomial, degree) from state-dependent decisions and operation status. The replacement takes the simpler route: it removes the current-best-degree gate and attempts the GCD stage for every admitted multiplier satisfying explicit static degree and resource limits. Its per-node duplicate set therefore refers to a fixed scheduled procedure, not to changing degree eligibility. It still does not retry a timed-out multiplier automatically, which remains a documented conservative limitation.

4.8 F08: candidate existence is not successful improvement

The original search may obtain a certified candidate, compare its cost with the incumbent, keep the incumbent unchanged, and then nevertheless take a success exit based on degree/history conditions. This is logically different from finding an improved expression. The outcome can be an unchanged answer despite unexplored candidates that might have helped.

The correction is to separate three events: an algebraic relation was found; an equal candidate was reconstructed; a strictly better admissible representation was accepted. Only the third should update the incumbent or trigger an improvement-specific stopping policy. The revised queue has no history-based success exit merely because a candidate exists.

This finding is a search-quality issue, not by itself an incorrect returned equality. The terminal certificate in the core often converts the consequence into a missed opportunity rather than a wrong answer.

4.9 F09: the final candidate, not its ancestor, needs certification

The numerical core first certifies orbit candidates and then applies a list of polishing transformations. It selects the cheapest polished variant without rechecking each variant at that point. Later certification of the selected overall result provides an important safety net for exact numeric inputs, but it can discard the whole attempt if the cheapest variant is unsuitable, even when another retained variant was valid.

A robust candidate pool should keep the unpolished certified expression and treat every polish as a new proposal. Filter proposals by certificate *before* ranking or replacing the incumbent. The new implementation follows that pattern, using distinct provenance labels such as `MultiplierOrbit` and `MultiplierOrbit/ToRadicals`.

The issue does not establish that `Simplify` or `ToRadicals` ordinarily returns wrong numeric values. It identifies an unnecessarily complicated proof boundary, especially when future heuristic polishing is added, and a failure mode for conservative loss of an already found result.

4.10 F10: numerical rejection is not an exact proof of inequality

The fixed routine rejects a candidate when a 30-digit numerical difference exceeds an absolute threshold. A fixed-precision computation is not a certified enclosure. Severe cancellation, internal precision limits, or a large expression's evaluation path can produce an unreliable estimate. The absolute threshold also ignores scale.

A numerical filter can safely *prioritize* candidates, or reject them when disjoint certified intervals prove inequality. It cannot be assumed to be an exact negative certificate solely because the inputs are exact. No native false rejection has been reproduced in this audit; this is a risk in the proof contract, not a measured defect rate.

The proposed acceptance routine removes the numerical filter. A future optimization should use directed or ball arithmetic with a proved nonzero enclosure, or conservatively fall through to exact normalization. It should never promote a close decimal match to a proof of equality.

4.11 F11: a local factorization guard does not prove the emitted rewrite

The common-factor routine tests a condition sufficient for distributing a principal power across an aggregate product FS . It then emits factors with individually multiplied exponents. These are not the same transformation. Positivity of the aggregate F does not make each extracted factor positive or justify every branch-sensitive power merge.

For the logical distinction, take $F = (-2)(-3) = 6$ and exponent $1/2$. The aggregate identity $\sqrt{F} = \sqrt{6}$ is harmless, but

$$\sqrt{-2} \sqrt{-3} = -\sqrt{6}.$$

Equivalently, an encoded base -1 with total exponent 2 has positive aggregate value 1 , yet replacing $((-1)^2)^{1/2}$ by $(-1)^1$ changes the value. These examples show that the *guard's inference* is insufficient for the whole emitted operation.

They are not claimed to be native end-to-end failures: evaluation and FactorList normalization may prevent a particular encoding from reaching the helper, and numerical Factorc results are subsequently certified. The exported symbolic Factorc path lacks that numerical certificate, so its scope deserves particular caution. A proof of reachability, or a native adversarial regression, is still needed before labeling this a demonstrated wrong automatic output.

The replacement removes this bespoke encoded factorizer. It offers an ordinary factor-terms candidate only within the common exact acceptance boundary. This trades some heuristic transformations for a much smaller branch-sensitive code surface.

4.12 F12–F16: remaining contract and coverage limitations

F12: output language. Counting Root objects discourages one form of hidden complexity but does not exclude all nonradical syntax. An exact-algebraic trigonometric expression returned by a custom solver can defeat a naive rational-power count. The revised grammar checks the candidate tree independently of the cost; a constant is not admitted merely because NumericQ happens to be true.

F13: specialized coverage. The quadratic fast path requires a rational square discriminant and therefore omits indirect fourth-root forms such as $\sqrt{4 + 3\sqrt{2}}$. It also does not provide a systematic cubic-in-quadratic path. A slow general search might still solve such examples, so the finding concerns the missing *fast path*, not a proven failure of every default call. Section 6 derives the added methods.

F14: independent subexpressions. Rejecting an entire mixed-precision host loses exact subexpressions that could be safely processed in isolation. Similarly, rolling back all ToRadicals conversions when any opaque root remains discards successful independent conversions. The replacement processes lists and arithmetic constituents under a single budget, leaving inexact numeric targets and unsupported heads unchanged. It converts opaque numeric nodes independently rather than demanding that the whole host convert at once.

F15: algebraic and expression growth. The trial count does not control root index, polynomial degree, coefficient bit length, or the size of a solved-root orbit. Syntactically different multipliers can be algebraically identical. The new queue canonicalizes admitted multiplier values for deduplication, caps its cardinality before insertion, bounds phase enumeration and candidate counts, and applies time/memory limits before expensive operations. A degree check after computing a minimal polynomial cannot prevent all expense of constructing that polynomial; this limitation remains explicit.

F16: observability. Returning the original expression conflates unsupported input, no useful candidate, exhaustion, timeout, failed equality proof, and a true impossibility theorem. Hard-coded factorization passes, recursive depth caps, prime-count thresholds, and prime-ratio thresholds further obscure the actual search. The replacement returns optional structured diagnostics and has

no user-visible marker representation. The original cost usage string should be corrected even in a minimal patch. Reproducible comparisons also require a source commit, kernel version, options, hardware, and cold/warm-cache policy, none of which can be inferred from an isolated successful identity.

5 Algebraic steps that should not be “fixed” incorrectly

A useful review also identifies suspicious-looking code that is justified. Overcorrecting it can introduce new branch bugs.

5.1 The direction of Order

Wolfram’s `Order[a,b]` returns 1 when a precedes b in canonical order, not -1 . Thus the original comparison against 1 is appropriate for its equal-length lists of nonnegative integer costs. Structural order is not a general numerical comparison for arbitrary expressions, but that is a different issue. The native suite includes a simple tuple-order regression.[13]

5.2 The nested-power guard has a valid core

For nonzero z , principal powers satisfy

$$(z^b)^c = z^{bc}$$

when $c \in \mathbb{Z}$. More generally this identity holds for arbitrary c when $b \in \mathbb{R}$ and $-1 < b \leq 1$. To see the latter, $\text{Arg } z \in (-\pi, \pi]$ implies $b \text{ Arg } z \in (-\pi, \pi]$, so no logarithm wrap occurs in $\text{Log}(z^b) = b \text{ Log } z$. For integer c , any logarithm wrap contributes $\exp(2\pi i k c) = 1$.

Thus the repaired code’s corresponding sufficient condition is not inherently wrong. Zero and undefined powers require their usual separate domain treatment. The real concern in F11 is a larger composed transformation not fully covered by that local guard.

5.3 The denominator formula has the correct sign

Let $d \neq 0$ be algebraic with minimal polynomial

$$p(X) = a_0 + a_1X + \cdots + a_nX^n.$$

Irreducibility and $d \neq 0$ imply $a_0 \neq 0$. Since $p(d) = 0$,

$$\frac{1}{d} = -\frac{a_1 + a_2d + \cdots + a_nd^{n-1}}{a_0}. \quad (4)$$

For the original polynomial quotient $q(X) = p(X)/(X - d)$, evaluating at zero gives $q(0) = -a_0/d$. Therefore $q(0)/(-a_0) = 1/d$: the sign in that code is correct.

The improvement is to bound the operation, preserve the nonzero-domain condition, validate the result, and use the coefficient formula directly where convenient. Merely changing its sign would break a valid identity. The standalone rationalization API in the proposal allows a certified result that is not cheaper, because denominator rationalization is a different objective from denesting.

5.4 Numerical scheduling is not automatically unsound

Some numerical tests in the marker wrapper decide which products to group or in which order to explore them. Grouping the same exact factors into the same exact product does not itself change their value. Those tests are weaker engineering choices for reproducibility and robustness, but they should not be confused with a numerical test that authorizes a mathematically different value. Each use of approximation must be assessed at its actual trust boundary.

6 Low-cost methods worth adding first

The following derivations provide concrete producers for the same certificate-gated engine. Their use does not depend on trusting a fragile pattern match or an unqualified general denesting theorem. The quadratic classification is classical and appears in BFHT and Gkioulekas; the cubic trace-norm criterion is consistent with Cavallo's restricted quadratic-field problem.[5, 7, 8]

6.1 Direct quadratic denesting

Suppose $a, b, c \in \mathbb{Q}$, $c > 0$, and $a + b\sqrt{c} > 0$. Put

$$\Delta = a^2 - b^2c.$$

When $\Delta = d^2$ with $d \in \mathbb{Q}_{\geq 0}$ and $a \geq d$, define

$$S = \sqrt{\frac{a+d}{2}} + \operatorname{sgn}(b)\sqrt{\frac{a-d}{2}}. \quad (5)$$

Both square-root arguments are nonnegative. Their sum is a and four times their product is b^2c . Expanding S^2 therefore gives $a + b\sqrt{c}$. Since the first square root is at least the second, $S \geq 0$, so S is the principal square root.

This sign argument is essential for $b < 0$:

$$\sqrt{3 - 2\sqrt{2}} = \sqrt{2} - 1,$$

not $1 - \sqrt{2}$. If the original real radicand is negative, apply the positive-radical method to its negative and multiply the resulting nonnegative root by i .

The proposed parser absorbs the nonrational summand's rational coefficient into its square. For instance, it may identify $15\sqrt{3}$ as $+\sqrt{675}$. This avoids brittle coefficient patterns in an orderless product. The equality $t = \operatorname{sgn}(t)\sqrt{t^2}$ is itself checked before using the extracted representation. It is a practical parser for a single nonrational summand, not a complete quadratic-field recognizer for arbitrary rational functions.

6.2 Indirect fourth-root denesting

Assume now $\Delta < 0$ and $a + b\sqrt{c} > 0$. Then $b > 0$. If

$$h = \sqrt{b^2 - a^2/c} \in \mathbb{Q},$$

put

$$S = c^{1/4} \left(\sqrt{\frac{b+h}{2}} + \operatorname{sgn}(a) \sqrt{\frac{b-h}{2}} \right). \quad (6)$$

Here $0 \leq h \leq b$, so the square-root arguments are nonnegative. Squaring gives

$$S^2 = \sqrt{c} \left(b + \operatorname{sgn}(a) \sqrt{b^2 - h^2} \right) = b\sqrt{c} + a.$$

The first root in the parentheses is at least the second; hence the selected expression is nonnegative. In the case $a = 0$, the formula is interpreted with $\operatorname{sgn}(0) = 0$ and the second root vanishes.

The rationality condition is equivalently $-c\Delta \in \mathbb{Q}^2$. As immediate tests,

$$\begin{aligned} \sqrt{4 + 3\sqrt{2}} &= 2^{1/4}(1 + \sqrt{2}), \\ \sqrt{-4 + 3\sqrt{2}} &= 2^{1/4}(\sqrt{2} - 1). \end{aligned}$$

These are useful regressions because the direct rational-discriminant path cannot produce them. Adding this small method before minimal-polynomial search is a substantial practical opportunity with a short proof and cheap exact arithmetic.

6.3 Complex quadratic constants

For $a, b \in \mathbb{Q}$, $b \neq 0$, and a rational $n = \sqrt{a^2 + b^2}$,

$$\sqrt{a + ib} = \sqrt{\frac{n+a}{2}} + i \operatorname{sgn}(b) \sqrt{\frac{n-a}{2}}.$$

The square is $a + ib$, the real part is nonnegative, and the imaginary part has the sign of b . The proposal includes this direct complex-constant candidate, again subject to the same certificate. An example is $\sqrt{3 + 4i} = 2 + i$. Native automatic evaluation may already simplify some such examples, so a benchmark must distinguish kernel preprocessing from the denester's own work.

6.4 A cube root inside a quadratic field

Let $c > 0$ be a nonsquare rational and $b \neq 0$. Seek a *real* cube root of $a + b\sqrt{c}$ in the form $u + v\sqrt{c}$, with $u, v \in \mathbb{Q}$. Define

$$q = u^2 - cv^2, \quad T = 2u.$$

Taking norms and comparing rational parts yields

$$q^3 = a^2 - b^2c, \quad T^3 - 3qT - 2a = 0. \quad (7)$$

The coefficient of $\sqrt{c}$ is

$$b = v(3u^2 + cv^2) = v(T^2 - q),$$

so reconstruction must use

$$u = \frac{T}{2}, \quad v = \frac{b}{T^2 - q}. \quad (8)$$

Conversely, suppose $q, T \in \mathbb{Q}$ satisfy (7). The identity

$$(T^3 - 3qT)^2 - 4q^3 = (T^2 - q)^2(T^2 - 4q) \quad (9)$$

gives

$$4b^2c = (T^2 - q)^2(T^2 - 4q).$$

In particular $T^2 - q \neq 0$. With (8), it follows that $cv^2 = u^2 - q$. The rational and irrational coefficients of $(u + v\sqrt{c})^3$ are then exactly a and b . The real cube function is injective, completing the reconstruction proof.

For implementation, test that the real cube root of the rational norm is rational, factor the rational cubic $X^3 - 3qX - 2a$, and inspect its linear factors. A factorization of a polynomial over $\mathbb{Q}$ is enough; enumerating all integer divisors is not logically required. The preprint's broader integer-factorization-equivalence assertion is not needed for this method and is not adopted here.[8]

A rational cube norm alone is insufficient: $a = b = 1, c = 2$ gives norm -1 , but $X^3 + 3X - 2$ has no rational root. This disproves the restricted quadratic-field reconstruction, not every conceivable radical representation in a larger output class.

Positive examples include

$$(26 + 15\sqrt{3})^{1/3} = 2 + \sqrt{3}, \quad (2 + \sqrt{5})^{1/3} = \frac{1 + \sqrt{5}}{2}.$$

For a negative real radicand, the reconstructed real cube root must not be returned as the principal complex cube root. The code enumerates its three phases and certifies against the original principal-power input.

6.5 Rational numerators and future field-specific producers

For an input $A^{p/q}$ with integer p and $q > 0$, a candidate for the selected q th root can be raised to the *integer* power p . This extends the specialized producers to exponents such as $3/2$ and $-1/2$ without assuming an unsafe distribution of fractional powers. The original value is still the reference for the final check.

Further improvements should be added as independent producers. Multiquadratic field bases can recognize dependencies missed by syntactic term grouping. Honsbeek's square-root-of-two-cube-roots criterion provides a particularly attractive next module: in its nondegenerate setting it leads to a rational quartic condition, not an unbounded numerical guess. The short polynomial identity

$$\left(-\frac{s^2}{2} + su + u^2\right)^2 - (u^3 - s^3)(1 + u) = \frac{s^4 + 4s^3 + 8u^3s - 4u^3}{4}$$

is verified in the companion script. The structural necessity hypotheses belong to the source theorem; the identity alone proves only the reconstruction when its conditions hold. This producer is *not* implemented in the present replacement.[9]

7 The proposed replacement: design and compatibility

7.1 Structure and trust boundaries

The new file uses the separate context `RadicalDenestImproved` so that it can be loaded alongside the original without overwriting it. The public names mirror the familiar entry points, but the implementation is a conservative redesign rather than a textual patch. The main path is

resolve controls $\longrightarrow$ bounded traversal $\longrightarrow$ candidate producers,
candidate producers $\longrightarrow$ acceptance $\longrightarrow$ committed pass.

A single resolved option association is shared dynamically through the request. There is no marker-to-solver substitution stage, and the old encoded factorizer and history stack are not reused.

For each eligible numeric node, the cheap quadratic-field producer runs first. Exact reduction is also tried, which can detect whole-expression cancellation; an opaque result is rejected as an output. An optional ordinary factor-terms candidate and a custom candidate follow. Opaque algebraic inputs can be converted with `ToRadicals`. When the automatic solver still has a nested target, the bounded multiplier queue is tried.

The rewrite deliberately does not apply global `Simplify` to a symbolic host. It also does not accept a custom result solely because a source function was named “solver.” These two boundaries remove the most serious wrapper-level ambiguity.

7.2 The queue and the units of its limits

For a single target node, the queue holds at most “MultiplierCap” distinct admitted multipliers, counting all initial and user-supplied entries. Canonical algebraic values are used to deduplicate rather than only syntax. A user-supplied multiplier list replaces automatic initialization; an empty list means no multiplier trials for that node.

Automatic initialization offers 1 and bounded simple complements of rational-power factors in expanded terms. After a trial, distinct discriminant primes can generate streamed prime-power multiples. This is not equivalent to exploring all products of those primes. Removing eager subset products makes the cap meaningful, but loses some original search paths. Comparisons should therefore report both coverage and resource behavior; no blanket claim that the rewrite denests every example the old scheduler did is made.

“MaxTrials” is shared across the *whole request*, including list elements and repeated passes. In contrast, the multiplier cap and orbit-candidate cap apply per numeric node. “MaxRootIndex” limits the reduction index and hence phase enumeration, not the grammar of every possible supplied candidate. “MaxDegree” filters a computed minimal polynomial before its GCD/reconstruction stages. The computation that discovers its degree is already protected by time and memory controls but is not made cheap by that later filter.

7.3 Option reference

Option	Default	Meaning
"TimeBudget"	30	Nonnegative finite request allowance; zero returns the original input.
"OperationTime"	5	Positive finite allowance for a costly producer call, clipped to remaining time.
"CertifyTime"	5	Positive finite allowance for a complete exact predicate/certificate call, clipped to remaining time.
"MemoryBudget"	268435456	Additional memory allowance in bytes for the constrained request body, subject to kernel semantics.
"MaxTrials"	100	Maximum multiplier trials across the entire request.
"MultiplierCap"	64	Maximum distinct queued multipliers for each numeric node.
"MaxCandidates"	128	Maximum orbit proposals per numeric node; cheap methods are separate.
"MaxDegree"	32	Maximum computed minimal-polynomial degree admitted to reconstruction.
"MaxRootIndex"	16	Maximum reduction index for multiplier search.
"MaxLeafCount"	20000	Input, candidate, and selected intermediate expression-size filter.
"AllLevels"	False	False: one bottom-up pass. True: up to MaxPasses decreasing passes.
"MaxPasses"	4	Positive integer upper bound on repeated traversal passes.
"Multipliers"	Automatic	Automatic bounded proposal stream, or a supplied list.
"Solver"	Automatic	Automatic queue, or a trusted callback producing one candidate per eligible numeric target. Cheap built-in producers remain active.
"Factor"	False	Enable an optional factor-terms candidate, not all mathematical factorization.
"Verbose"	False	Emit diagnostics without redefining system output functions.
"DetailedResult"	False	Return an association containing expression, outcome, counters, and certificates.

The public `Strad`, `DenestRadicals`, and `DenestCore` defaults are resolved independently, so `SetOptions` on a given entry point has a defined effect. A legacy positional Boolean is supported by the two wrapper names. It is translated into the explicit pass policy above; it does not reproduce the old marker-resolution schedule.

7.4 Commit and rollback policy

A traversal pass can find several local certified improvements. The engine commits only after the entire pass completes, the aggregate cost decreases, and an additional whole-value certificate

succeeds when the whole input is numeric algebraic. A symbolic arithmetic or list host relies on the local certificates and congruence argument.

The committed expression and its log are stored as a single record, so the code does not separately assign a new expression and only later attach its supporting audit data. A timeout or memory failure during a later pass returns the last committed pass, not an arbitrary partially rewritten tree. This is conservative: a good local improvement discovered during a pass that later times out can be lost. Retaining partial progress safely would require path-aware transactional commits, not merely exposing an internal partially built expression.

The log records the target, candidate, producer label, verifier name, and an exact zero residual. These are *recheckable CAS audit records*, not standalone proof objects accepted by a small independent theorem-prover kernel. Replaying RootReduce still trusts Wolfram's exact algebraic arithmetic. An independently checkable field-basis and root-isolation certificate is a worthwhile future enhancement.

7.5 Result interpretation and usage

A typical intended call is:

```
Get["code/StradImproved.wl"];
r = RadicalDenestImproved`Strad[
  x + Sqrt[4 + 3 Sqrt[2]],
  "AllLevels" -> True, "MaxTrials" -> 20,
  "TimeBudget" -> 10, "DetailedResult" -> True
];
r["Expression"]
r["Certificates"]
```

These are usage instructions, not an executed native transcript. The outcome is Improved or Unchanged; standalone denominator rationalization instead uses Rewritten for a changed certified result. Search status records request completion or the outer stopping reason, while separate fields identify operation timeouts, trial-limit exhaustion, and degree skips.

SearchStatus -> "Completed" means the scheduled bounded procedure ended normally. It does *not* mean that all mathematical possibilities were examined; per-operation timeouts and exhausted finite queues are reported separately. An unchanged answer is never a theorem that no denesting exists.

The proposal does not yet have a complete public assumptions language, full symbolic field arithmetic, a general Honsbeek module, arbitrary complex quadratic-field parsing, a beam search through temporarily worse forms, or an independently verified proof-certificate format. These are limitations of the supplied code, not features deferred invisibly behind an optimistic description.

8 Verification, regression tests, and reproducibility

8.1 What actually ran

The independent verification script ran under Python 3.13.5 and SymPy 1.14.0. Its final recorded run passed all 33 named checks. The parameterized portions cover 144 direct-quadratic cases, 108 indirect-fourth-root cases, and 60 cubic-in-quadratic cases, for 312 exact rational parameter cases in total. The JSON file records the scope and individual results.

These checks verify algebraic identities, exact coefficient conditions, branch-sign witnesses, the Ramanujan minimal polynomial and field GCD, the cap counterexamples, a model of the new insertion invariant, the changing degree gate, and the denominator inverse identity. They do *not* verify that the Wolfram parser reads every intended pattern the same way, that package contexts resolve on a native kernel, or that the Wolfram scheduler and timer semantics match the design.

The supplied string/comment-aware scanner confirms matched delimiters in the generated .wl, .wlt, and .wls files. It recognizes nested Wolfram comments and escaped strings. It is intentionally labeled a lexical check, not a native parser, type checker, or execution substitute.

8.2 Native integration suite: pending, not passed

The tests/StradImproved.wlt suite contains 43 verification tests. They cover direct and indirect examples, principal negative cube roots, rational exponent numerators, custom-solver rejection in symbolic hosts, inexact proposals, grammar restrictions, exact equality, assumptions isolation, held and unknown heads, mixed lists, zero budgets, invalid options, default resolution, shared trial counters, queue bounds, certificate replay, and forced-multiplier reconstruction.

The suite contains both *soundness tests* and *capability acceptance tests*. A capability test can fail because a conservative operation timed out or a representation was not reconstructed. That is still a test failure to investigate; it should not be hidden by weakening every assertion to “output equals input.” Conversely, an assertion that an arbitrary hard radical must denest under all machines’ small time budgets would be an unreasonable universal promise.

A future native run should start in a fresh kernel, load both implementations only in separate contexts, and record the exact source hashes. The runner writes the full native test report plus metadata with \$Version, \$SystemID, and the new source hash. It checks the current documented ReportSucceeded property, with an older-property fallback, instead of assuming an unverified pass count.[14]

```
# From the archive root, with a licensed/configured Wolfram kernel:
wolframscript -file tests/run_tests.wls

# Independent checks executed for this review:
python verification/verify_mathematics.py
python verification/check_wl_structure.py

# Optional reproductions against a separately obtained reviewed source:
wolframscript -file tests/original_regressions.wls /path/to/StradFixed.wl
```

Both attempts to reach the available Wolfram evaluator failed with a connector-level HTTP 404 response. There was no local Wolfram kernel. No timing or output in this report should be read as a native StradFixed or StradImproved benchmark. The inability to replay the preceding review’s logs is separately recorded; it is not concealed by the successful independent mathematics checks.

8.3 A benchmark corpus that distinguishes tasks

A useful comparison corpus should contain easy direct roots, indirect roots, imaginary and complex branches, cubic-in-quadratic successes and failures, pure-cubic identities, multiquadratic inputs, partial denestings, global cancellations, and malformed or unsupported hosts. Each example should have an expected *value* and an expected *capability class*, not just a preferred printed string.

For instance,

$$\begin{aligned}\sqrt{112 + 70\sqrt{2} + (46 + 34\sqrt{2})\sqrt{5}} &= 5 + 4\sqrt{2} + 3\sqrt{5} + \sqrt{10}, \\ \sqrt{16 - 2\sqrt{29} + 2\sqrt{55 - 10\sqrt{29}}} &= \sqrt{5} + \sqrt{11 - 2\sqrt{29}}\end{aligned}$$

exercise dependencies and partial reduction. Their squared identities and positivity are checked independently here. They are not asserted to be covered by the new single-quadratic-summand parser. A whole-expression cancellation case can require a producer that looks beyond individual roots.

For resource experiments, record median and tail latency, maximum memory, number of polynomial constructions, maximum admitted degree, candidate rejection reasons, and output cost. Separate cold and warm caches. A meaningful improvement would be fewer wrong-value pathways and better bounded behavior, even before claiming a higher success rate. Do not rank algorithms by one attractive identity or assume that a smaller internal field degree implies a better printed expression.

9 A practical improvement roadmap

9.1 First: establish a trustworthy acceptance boundary

For an incremental patch to the existing code, the first priorities are local certification of every custom-solver result, certification after every polish, and an explicit host/assumptions policy. Keep the original selected value available throughout the transformation. Do not let output from a plugin or callback bypass the candidate interface. These changes can be made before replacing the entire scheduler.

Next, wrap the full request and every costly producer, validate all controls, propagate resolved defaults, and enforce queue limits *before* constructing batches. Treat list and recursive budgets as part of the public API. Add the cap-7 helper reproduction and the symbolic-host zero-solver reproduction as permanent regression tests.

9.2 Second: improve algorithms without widening the trust boundary

The fourth-root and cubic-in-quadratic producers are small enough to review mathematically. More ambitious producers can use a common number-field representation, multiquadratic squarehood tests, and special pure-cubic reconstruction. Their only authority should be to propose candidates. The exact acceptance boundary remains unchanged.

A better multiplier scheduler can retain the original heuristic's useful combinations without eager allocation. Use a priority queue of lazily generated exponent vectors, a reproducible ranking, and a

cap on *generated* states as well as retained states. Cache invariant polynomial data separately from trial status. A timed-out GCD should not be indistinguishable from a proved constant GCD.

Field-degree estimates, coefficient-height estimates, and subfield information can rank work. They should not suppress a branch as mathematically impossible unless a theorem with matching hypotheses establishes that conclusion. A bounded beam search can occasionally accept temporary growth, but it needs its own state budget and a final strict-improvement commit rule.

9.3 Third: make results independently auditable

The long-term certificate format should record a primitive element or a compatible tower, polynomial relations, nonzero denominators, and isolating intervals or regions for selected roots. An independent checker could then verify identities without rerunning the whole heuristic. This is more ambitious than recording an exact zero from the same CAS, but it is the natural direction when denesting is part of formal or reproducible mathematical research.

A continuous-integration matrix should exercise supported Wolfram versions and platforms, run mutation tests on the candidate producers, and monitor operation-level budget failures. It should also preserve known *negative scoped results*: failure of the cubic-in-quadratic criterion belongs to that field class, while a search timeout belongs only to that run. Those labels should never collapse into a single “cannot denest” message.

10 Conclusion

The corrected implementation substantially improves on an unguarded radical manipulator, particularly in its original-value branch certification. Its remaining weaknesses are concentrated at interfaces: the symbolic wrapper can trust an arbitrary custom result; resource controls miss the operations that dominate cost; and search history does not cleanly distinguish algebraic data, attempted stages, certified candidates, and actual improvements.

The proposed replacement addresses those boundaries in a smaller, more conservative design and includes concrete low-cost producers justified by exact identities. It intentionally gives up some scheduling behavior and unsupported symbolic transformations. The independent checks substantiate the mathematical formulas and finite counterexamples, while the native regression suite remains pending. The defensible deliverable is therefore an actionable reviewed design, executable proposed source, explicit tests, and a transparent validation record—not an unsupported assertion that a new universal denester has been completed or that every source-level concern has already been reproduced on a Wolfram kernel.

A Archive inventory and build instructions

The archive contains `review.tex`, its compiled `review.pdf`, the standalone source under `code/`, native tests and runners under `tests/`, independent check scripts and recorded JSON results under `verification/`, a README, a source-access ledger, and a checksum manifest. No third-party literature PDFs or font files are redistributed.

The TeX source is self-contained with an embedded bibliography and no external figures. A standard TeX Live installation with New PX, Inconsolata, listings, and tcolorbox can rebuild it:

```
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
```

The Python scripts overwrite their own result JSON files on a successful rerun. Native test-report files are not fabricated in the distributed archive; `native_status.json` records the pending state. Running the supplied native runner creates separate native outputs.

B Targeted literature and source references

The references below identify the inspected implementation and the small subset of the much larger literature most directly relevant to this review. Repository references are mutable main URLs accessed on the audit date. The prior-review directory is cited only for its accessible README/listing, not as a claim that its body or experiment logs were independently rechecked.

References

- [1] Vladimir Reshetnikov, repository *RadicalDenest*, `src/corrected/StradFixed.wl`, public raw rendering, accessed 5 September 2026.
<https://raw.githubusercontent.com/VladimirReshetnikov/RadicalDenest/main/src/corrected/StradFixed.wl>.
- [2] *Radical Denesting: Unified Research Report*, current TeX rendering in the same repository, accessed 5 September 2026. Relevant technical sections consulted.
https://raw.githubusercontent.com/VladimirReshetnikov/RadicalDenest/main/docs/report/radical_denesting_unified.tex.
- [3] *RadicalDenest literature collection*, repository directory and README, accessed 5 September 2026. Directory-level inspection; not a complete verification of its archived assets.
<https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/docs/literature>.
- [4] *Unified code review*, repository directory and README, accessed 5 September 2026. Body-file and detailed-log retrieval unsuccessful in this audit.
<https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/src/code-review/unified>.
- [5] A. Borodin, R. Fagin, J. E. Hopcroft, and M. Tompa, “Decreasing the nesting depth of expressions involving square roots,” *Journal of Symbolic Computation* **1**(2), 169–188, 1985. DOI: [10.1016/S0747-7171\(85\)80013-4](https://doi.org/10.1016/S0747-7171(85)80013-4).
<https://www.cs.toronto.edu/~bor/Papers/decreasing-nesting-depth-for-expressions.pdf>.
- [6] D. J. Jeffrey and A. D. Rich, “Simplifying square roots of square roots by denesting,” in M. J. Wester (ed.), *Computer Algebra Systems: A Practical Guide*, Wiley, 1999, pp. 61–72. Accessible chapter copy, especially the practical measure and recursive-method discussion.
<https://cybertester.com/data/denest.pdf>.
- [7] E. Gkioulekas, “On the denesting of nested square roots,” *International Journal of Mathematical Education in Science and Technology* **48**(6), 942–953, 2017. DOI: [10.1080/0020739X.2017.1290831](https://doi.org/10.1080/0020739X.2017.1290831). Author manuscript,

direct and indirect denesting theorems.

<https://faculty.utrgv.edu/eleftherios.gkioulekas/papers/submitted/denesting-square-roots.pdf>.

- [8] A. Cavallo, *Denesting cubic radicals*, arXiv:2403.04776, version 2, 2024. The restricted quadratic-field criterion is relevant here; the paper's integer-factorization-equivalence assertion is not used.
<https://arxiv.org/html/2403.04776v2>.
- [9] M. Honsbeek, *Radical Extensions and Galois Groups*, doctoral dissertation, Radboud University Nijmegen, 2005. Selected passages in Chapters 3–4 on square roots of sums of cube roots and depth-one field criteria.
https://www.math.ru.nl/~bosma/students/honsbeek/M_Honsbeek_thesis.pdf.
- [10] Wolfram Research, *RootReduce*, Wolfram Language documentation, accessed 5 September 2026.
<https://reference.wolfram.com/language/ref/RootReduce.html>.
- [11] Wolfram Research, *TimeConstrained*, Wolfram Language documentation, accessed 5 September 2026.
<https://reference.wolfram.com/language/ref/TimeConstrained.html>.
- [12] Wolfram Research, *MemoryConstrained*, Wolfram Language documentation, accessed 5 September 2026.
<https://reference.wolfram.com/language/ref/MemoryConstrained.html>.
- [13] Wolfram Research, *Order*, Wolfram Language documentation, accessed 5 September 2026.
<https://reference.wolfram.com/language/ref/Order.html>.
- [14] Wolfram Research, *TestReportObject*, Wolfram Language documentation, accessed 5 September 2026.
<https://reference.wolfram.com/language/ref/TestReportObject.html>.